

LIVEKIT + FULLY OFFLINE MODELS

Building a near-human conversation buddy with custom knowledge, RAG, memory and MCP

Latest architecture and implementation guide - verified against current LiveKit documentation and current local model catalogs as of 17 September 2026.

Core design principle: LiveKit owns realtime media and conversation orchestration; local models own speech and reasoning; RAG owns knowledge; memory owns continuity; MCP owns actions/capabilities.

Scope: self-hosted LiveKit, no LiveKit Cloud, no LiveKit Inference, local VAD/turn detection, local STT, local LLM, local TTS, local embeddings/RAG, local MCP servers, and an extensible custom-data attachment pipeline.

What this guide updates from the supplied PDF

- Keeps the supplied PDF's core four-pillar offline idea: Silero VAD, local Whisper-family STT, Ollama/vLLM for the LLM, and Kokoro for local TTS. The supplied PDF also proposes a multi-container environment and a Python LiveKit agent.
- Updates the LiveKit API shape to the current AgentServer + AgentSession architecture and current MCPToolset pattern.
- Adds the current audio turn detector, including the locally runnable v1-mini option, so the system can remain offline without relying on LiveKit Inference.
- Adds current local model choices including Qwen3.5/Qwen3.6/Qwen3.8, Gemma 4, Qwen3-ASR, Parakeet/NVIDIA NeMo, Kokoro, Piper, Chatterbox and F5-TTS.
- Adds a production-grade custom-data attachment model: documents, PDFs, scanned pages, audio, video transcripts, code, notes and structured data can all feed one knowledge plane while remaining distinguishable by source and provenance.

Important interpretation of "100% offline": initial model/package downloads still need internet, but after models and dependencies are present locally, runtime inference, retrieval, tool execution and LiveKit media orchestration can stay on the local machine or private LAN. A browser on another device still uses the local network; a public remote client is not network-offline.

Contents

1. Target system and design goals
2. Current LiveKit architecture
3. The complete offline model stack
4. Local model catalog and integration paths
5. Custom-data attachment and RAG pipeline
6. Memory design for a persistent buddy
7. MCP and safe tool use
8. Near-human conversation behavior
9. Reference implementation skeleton

- 10. Local deployment and startup
- 11. Hardware planning and model-sizing
- 12. Testing, evaluation and latency instrumentation
- 13. Migration from the supplied PDF
- 14. Build roadmap
- 15. Troubleshooting matrix
- Appendix A. Example schemas
- Appendix B. Official source index

1. Target system and design goals

The goal is not a text chatbot with speech added. It is an always-listening conversational system that can talk naturally, pause, be interrupted, remember earlier interactions, answer from attached private knowledge, and invoke controlled capabilities when an action is required.

Goal	What the system must do	Architecture implication
Natural turn-taking	Wait through meaningful pauses; detect actual end of thought; recover from false interruptions.	Silero VAD + LiveKit audio turn detector v1-mini + adaptive interruption settings.
Low perceived latency	Start useful work before the turn is fully closed; avoid silence during tools/RAG.	Preemptive generation, streaming STT/TTS, short filler speech for long operations.
Private knowledge	Answer from attached data without retraining the LLM.	Separate ingestion and retrieval plane; inject context before generation.
Persistent memory	Remember selected facts and episodic events without bloating the prompt.	Working memory + episodic memory + explicit profile memory.
Actions	Use calendar/tasks/files/etc. only when needed.	MCPToolset + filtered tool exposure + bounded tool steps.
Offline	No cloud model dependency at runtime.	Self-hosted LiveKit + local model servers + local storage + local MCP.

The separation that keeps the design sane

REALTIME	= LiveKit/WebRTC + AgentSession
SPEECH	= VAD + STT + TTS
BRAIN	= local LLM
KNOWLEDGE	= RAG / knowledge graph / structured store
MEMORY	= episodic + profile + working context
ACTIONS	= MCP + function tools
OBSERVABILITY	= timestamps, transcripts, tool traces, model metrics

Do not conflate RAG and MCP: RAG answers "what do I know that is relevant?" MCP answers "what can I do or query right now?" Retrieval generally belongs before the LLM response; actions belong behind explicit tools.

2. Current LiveKit architecture

Current LiveKit Agents uses AgentServer to manage jobs and AgentSession to orchestrate the realtime conversation. The official Python starter project as of September 2026 uses the AgentServer/AgentSession pattern, and the current release shown by the repository is livekit-agents 1.8.2.

Layer	Current LiveKit concept	Offline configuration
-------	-------------------------	-----------------------

Transport	LiveKit Server / WebRTC	Self-host LiveKit Server locally or on private infrastructure.
Worker	AgentServer	Run the Python agent locally with dev/start modes.
Session	AgentSession	Instantiate STT, LLM, TTS, VAD, turn handling and tools.
Turn handling	TurnHandlingOptions + TurnDetector	Pin audio turn detector to v1-mini for fully local runtime.
RAG hook	on_user_turn_completed()	Query local indexes and add only relevant context.
Tools	Function tools + Toolset + MCPToolset	Run local MCP servers via stdio or local Streamable HTTP.
Scaling	Job distribution / worker pool	Optional later; one local agent process is sufficient for development.

Current realtime path

```

Microphone
-> WebRTC / LiveKit Room
-> AgentSession
  -> Silero VAD
  -> local audio turn detector (v1-mini)
  -> local STT
  -> on_user_turn_completed()
    -> hybrid RAG + memory retrieval
    -> context injection
  -> local LLM
    -> direct response OR MCP/function tool
  -> streaming local TTS
-> WebRTC audio back to user

```

Two pipeline families

Pipeline	Strength	Use in this project
STT -> LLM -> TTS	Inspectable; strong tool calling; easiest RAG insertion; granular metrics; separate components can be swapped.	Primary architecture for the first complete offline buddy.
Realtime / speech-to-speech	Potentially lower latency and more overlapping speech behavior.	Secondary experiment after RAG/MCP pipeline is stable; current full-duplex options are newer and less suitable as the first offline foundation.

3. The complete offline model stack

A fully local buddy is a system of cooperating models, not one model. The supplied PDF identifies the four main voice pillars. The current architecture adds turn detection, embeddings/reranking, and explicit memory/tool layers around those pillars.

Function	Baseline from supplied PDF	Current practical offline choices	LiveKit integration
VAD	Silero VAD	Silero VAD; custom energy/WebRTC VAD can be a fallback.	Official local plugin; CPU-friendly.
Semantic turn detection	Not separated in the PDF	LiveKit audio TurnDetector v1-mini.	Built into current Agents; v1-mini runs locally on CPU.
STT	Whisper / whisper.cpp	Qwen3-ASR 0.6B/1.7B; Whisper large-v3 / faster-whisper / whisper.cpp;	Use a native plugin where available; otherwise wrap as local HTTP or implement a

		NVIDIA Parakeet/Nemotron family; Moonshine.	custom STT node. Non-streaming Whisper-style services need VAD + StreamAdapter.
LLM	Ollama / vLLM; Llama/Mistral examples	Qwen3.5, Qwen3.6, Qwen3.8, Gemma 4, Mistral Small 3.2, other Ollama/vLLM-served open weights.	OpenAI-compatible local endpoint or direct local plugin.
TTS	Kokoro 82M	Kokoro; Piper; Chatterbox Turbo/Nano/V3; F5-TTS; XTTS-family.	Kokoro has an official LiveKit local integration via Kokoro-FastAPI. Other engines use a local wrapper/custom TTS node.
Embedding	Not covered in PDF	Qwen3-Embedding 0.6B/4B/8B; other local embedding models.	Outside LiveKit core; connect via your RAG service.
Reranker	Not covered in PDF	Qwen3-Reranker 0.6B/4B/8B or another local cross-encoder.	Outside LiveKit core; use during retrieval.
Memory	Not covered in PDF	PostgreSQL/pgvector + structured tables; optional graph store.	Called from RAG hook or helper service.
MCP	Not covered in PDF	Local stdio MCP servers or local Streamable HTTP.	Official MCPToolset in Python.

Offline boundary checklist

- LiveKit Server URL is local/private, not a LiveKit Cloud URL.
- Agent does not instantiate inference.* models that require LiveKit Inference.
- TurnDetector is explicitly v1-mini if you want to guarantee the turn model itself is local.
- STT endpoint is localhost/private LAN.
- LLM endpoint is localhost/private LAN.
- TTS endpoint is localhost/private LAN.
- Embedding and reranking models are local.
- MCP servers are local processes or private services.
- RAG storage is local PostgreSQL/pgvector or another local store.
- Browser app, token service and static assets are served locally for a completely private LAN deployment.

4. Local model catalog and integration paths

This section is intentionally a catalog rather than a single winner. Model choice depends on the hardware, languages, concurrency and voice style you need. File sizes below are representative Ollama package sizes where the official catalog exposes them; actual runtime memory is higher because of model runtime state, KV cache and concurrent services.

4.1 LLM candidates

Family / model	Local availability observed	Notable traits relevant to buddy	Typical integration
Qwen3.5 4B / 9B	Ollama: 3.4GB / 6.6GB	Multimodal family; 256K context in catalog; available in small local sizes.	Ollama -> LiveKit OpenAI-compatible LLM.
Qwen3.5 27B / 35B	Ollama: 17GB / 24GB	Larger local reasoning capacity; 256K context in catalog.	Ollama or vLLM.

Qwen3.6 27B / 35B	Ollama: ~18GB / 23GB	Current family focused on agentic coding/thinking preservation; 256K context.	Ollama or vLLM.
Qwen3.8 27B	Ollama: ~18GB; 256K context	Current 27B local option with reported agentic-task improvements.	Ollama; vLLM where model support is available.
Gemma 4 E2B / E4B	Ollama: 7.2GB / 9.6GB	Edge-oriented sizes; multimodal; designed for reasoning and agentic workflows.	Ollama.
Gemma 4 12B / 26B / 31B	Ollama: 7.6GB / 19GB / 20GB	Workstation sizes; large context in current catalog.	Ollama or vLLM.
Mistral Small 3.2 24B	Ollama: ~15GB	Native function calling emphasis and reduced repetition vs earlier release.	Ollama / vLLM.
Qwen3 8B / 14B / 30B / 32B	Ollama: ~5.2GB / 9.3GB / 19GB / 20GB	Mature family; official Qwen docs describe function calling and multi-step tool use.	Ollama / vLLM.

For a conversational buddy, test at least three LLM tiers

- Small/fast tier: Qwen3.5 4B or Gemma 4 E2B/E4B. Use it when immediate response is more important than long reasoning.
- Middle tier: Qwen3.5 9B, Gemma 4 12B or Qwen3 14B. Use it as the day-to-day local baseline for RAG + tools.
- Large local tier: Qwen3.8 27B, Qwen3.6 27B/35B, Gemma 4 26B/31B. Use it for deeper answers, harder tool planning or code-heavy knowledge tasks; expect substantially more hardware pressure.

Do not confuse context-window size with useful conversational capacity: A 256K context window does not mean you should keep 256K tokens of conversation alive. Keep the live prompt small, use retrieval, and summarize older conversation turns.

4.2 STT candidates

Model family	What current sources establish	Streaming note	Buddy integration
Qwen3-ASR 0.6B / 1.7B	Official Qwen3-ASR release supports language identification and ASR for 52 languages/dialects; docs include streaming and vLLM deployment paths.	Designed with streaming inference support.	Good candidate for a local streaming STT microservice.
Whisper family	The supplied PDF identifies local Whisper/whisper.cpp; LiveKit docs show Whisper-style STT may need VAD + StreamAdapter because some Whisper endpoints are not inherently streaming.	Usually buffered/chunked unless using a streaming wrapper.	Use local OpenAI-compatible transcription endpoint or custom STT.
NVIDIA Parakeet / NeMo Speech	Current NeMo Speech docs expose Parakeet, cache-aware streaming Conformer models, timestamps and word boosting.	Streaming capability depends on model family; Parakeet TDT v3 is documented as full-utterance in NeMo-Speech.cpp while other families provide streaming.	Excellent research path when NVIDIA GPU is available; wrap the exact model server API.
Moonshine	Small local ASR models exist, including English model cards from Useful Sensors.	Lightweight local option; language coverage narrower in the base model.	Custom STT adapter.

4.3 TTS candidates

Engine	Current evidence	Human-like potential	LiveKit integration path
Kokoro	LiveKit has a current official local plugin guide using Kokoro-FastAPI and OpenAI-compatible TTS endpoint.	Clean local baseline; simple deployment.	Official OpenAI TTS adapter -> local Kokoro-FastAPI.
Piper	Open Home Foundation project; current repo shows v1.4.2 and ongoing 2026 work including low-latency streaming synthesis and CUDA support.	Very fast/lightweight; voice realism depends on chosen voice.	Local HTTP/service wrapper or custom TTS node; licensing must be reviewed for your use.
Chatterbox Turbo / Nano / Multilingual V3	Official repo documents 350M Turbo for low-latency English, 110M Nano for CPU/on-device use, and 500M Multilingual V3 supporting 23+ languages including Hindi.	Strong option when expressive/paralinguistic speech and voice similarity matter.	Local service or custom TTS adapter; not the same official turnkey LiveKit plugin path as Kokoro.
F5-TTS	Active 2026 open-source repo with current releases and local inference workflows.	Natural speech and voice cloning experiments.	Custom local HTTP or TTS node.

4.4 RAG embedding and reranking candidates

Model	Current published shape	Use
Qwen3-Embedding-0.6B	0.6B parameters, 32K sequence, 1024-d embedding, instruction aware, MRL support.	Default local embedding baseline for a modest machine.
Qwen3-Embedding-4B	4B parameters, 32K sequence, 2560-d embedding, instruction aware.	Higher-capacity retrieval where hardware allows.
Qwen3-Embedding-8B	8B parameters, 32K sequence, 4096-d embedding, instruction aware.	Large local retrieval model for demanding corpora.
Qwen3-Reranker-0.6B / 4B / 8B	Matching reranker families are published.	Second-stage relevance ranking after vector/BM25 retrieval.

5. Custom-data attachment and RAG pipeline

"Attach data to the buddy" should mean adding a private knowledge collection linked to a persona or user, not fine-tuning the voice LLM for every upload. The data plane should be independently ingestible, versioned and removable.

UPLOAD

- > file fingerprint + access policy
- > parser / OCR / ASR / code parser
- > normalized documents
- > structure-aware chunking
- > metadata + provenance
- > local embeddings
- > vector index + full-text index
- > optional reranker
- > optional knowledge graph
- > retrieval API
- > LiveKit on_user_turn_completed()
- > prompt context
- > local LLM

Supported attachment families

Input	Ingestion path	Metadata to preserve
PDF / DOCX / TXT / Markdown	Text extraction; OCR when page is scanned; heading-aware chunking.	filename, page, section, title, source hash.
Scanned PDF	Rasterize page -> OCR -> layout-aware cleanup -> chunk.	page number, OCR confidence if available, bounding-box references if used.
Audio	Local ASR -> timestamped transcript -> speaker/segment metadata.	start_ms, end_ms, speaker, source file.
Video	Extract audio -> ASR; optionally sample frames and OCR/caption visual context.	timestamp, chapter/scene, frame refs.
Codebase	Language-aware parser -> symbol/documentation chunks -> file path and symbol metadata.	repo, commit, file path, symbol, line range, language.
CSV / spreadsheet	Schema extraction plus row/column-aware textualization; keep original structured store when exact arithmetic/filtering is needed.	sheet/table, columns, row key, data types.
Web snapshots / notes	Clean HTML/text -> chunk -> source metadata.	URL or origin note, capture date, title.

Recommended retrieval strategy

```

query = latest_user_turn

1) classify intent
   casual | knowledge | memory | action | mixed

2) retrieve in parallel
   semantic vector search -> top 20
   full-text/BM25 search -> top 20
   memory search -> top 10

3) merge + deduplicate

4) rerank -> top 6-10 evidence chunks

5) inject a compact evidence block
   source title + location + excerpt + source_id

6) LLM answers in natural speech

7) optionally cite/mention source in UI, not as a spoken bibliography

```

Critical RAG rule: retrieval should precede generation

The current LiveKit external-data guidance uses `on_user_turn_completed()` as the place to fetch relevant data and add it to the turn context before the LLM responds. That avoids making every knowledge lookup into a second model/tool round trip.

Evidence-aware context format

```

[PRIVATE KNOWLEDGE]
Source: Cloud architecture notes.md
Location: section "Agent orchestration"
Relevance: 0.91
Excerpt: ...
Source ID: src_8f3a
[/PRIVATE KNOWLEDGE]

[MEMORY]

```

Episode: 2026-09-12 conversation

Fact: User preferred local-first tooling for this project.

[/MEMORY]

Keep private sources distinguishable: Do not mix an old conversation memory, a source document, and a live tool response into one undifferentiated text blob. Label them so the LLM can reason about freshness and authority.

6. Memory design for a persistent buddy

A buddy feels persistent when it remembers the right things rather than when it stores every utterance forever. Use three memory tiers.

Tier	Lifetime	Examples	Storage
Working context	Current session	Last several turns, current task, active tool results.	AgentSession ChatContext / runtime object.
Episodic memory	Weeks to years	"We discussed migrating the app to Postgres." Conversation events and outcomes.	PostgreSQL rows + embedding + timestamp.
Explicit profile memory	Until changed/deleted	Language preference, preferred response style, recurring project names.	Structured PostgreSQL table with provenance and user controls.
Knowledge sources	Until revoked	Uploaded PDFs, code, recordings, notes.	Document store + chunk tables + vector/full-text index.
Live state	Very short-lived	Current calendar availability, task status, sensor state.	MCP/tool responses; do not persist automatically unless explicitly needed.

Memory extraction should be conservative

- Prefer explicit statements and repeated stable facts over one-off conversational guesses.
- Store a memory with source, timestamp and confidence/validation status.
- Let the user inspect, correct and delete durable memories.
- Do not store every utterance as a permanent memory. Keep raw transcripts separately if you need auditability.
- Use memory retrieval only when it can plausibly change the answer or conversational continuity.

7. MCP and safe tool use

Current LiveKit Agents has first-class MCP support in Python through MCPToolset. Current docs describe HTTP MCP servers and local stdio MCP servers, and they recommend filtering tool exposure when a server has many tools. Streamable HTTP is the modern transport; SSE is being deprecated in the MCP documentation.

Suggested local MCP topology

```
Agent
|
+-- MCPToolset: calendar
|   +-- list_events
|   +-- create_event
|
+-- MCPToolset: tasks
|   +-- list_tasks
|   +-- add_task
```

```

|
+-- MCPToolset: filesystem
|   +-- read_allowed_file
|
+-- MCPToolset: project-data
|   +-- search_project
|   +-- get_record

```

Rule	Implementation
Expose few tools	Use <code>allowed_tools</code> or toolset-level filtering. Prefer focused toolsets per capability.
Bound tool loops	Set a deliberate max tool-step budget. Do not solve infinite loops by simply increasing the limit.
Separate read vs write	Use different tool names and permission paths for retrieval and mutation.
Confirm irreversible actions	Deleting, sending, purchasing, publishing or changing live data should have a confirmation gate.
Summarize large results	Transform or truncate huge MCP results before they enter the LLM context.
Use async/background tools	Long operations should run without forcing the conversational pipeline into a silent dead wait.
Give progress speech	For long tools, a short filler/status utterance can make the wait feel intentional.

Tool decision flow

USER UTTERANCE

```

|
+--> Can answer from current context? --> YES --> speak
|
+--> Needs private knowledge? -----> RAG first
|
+--> Needs live external state/action? -> MCP/function tool
|
+--> Tool mutates live state? -----> confirm if required
|
+--> Tool result returned -----> compact result
|
+--> LLM
+--> TTS

```

8. Near-human conversation behavior

Voice realism is mostly a systems problem: timing, turn boundaries, interruption recovery, prosody and conversational restraint matter as much as model quality.

8.1 Turn-taking

Current LiveKit documentation recommends the audio turn detector as the default turn detector for the STT-LLM-TTS pipeline. The audio model combines acoustic and semantic cues, and the v1-mini model runs locally on CPU. It supports 14 languages, including English and Hindi. This is the key piece that lets the offline architecture avoid cloud turn inference.

Starting behavior to test

Situation	Expected buddy behavior
"I need to think for a second..." + silence	Wait; do not seize the turn.
"Yeah, exactly." as a backchannel	Avoid treating a short backchannel as a full new task unless context requires it.
User talks over TTS	Stop or reduce TTS quickly; preserve the user turn.

User self-corrects	Discard the stale planned answer when the new turn changes intent.
Long tool call	Speak a brief status cue, then continue when result arrives.
Very long user monologue	Use user-turn limits only as a safety/UX boundary; do not arbitrarily interrupt normal thought.

8.2 Preemptive generation

Current turn-handling options include preemptive generation, which allows the LLM to begin work before the turn is fully confirmed. This can reduce response delay but may waste generation when the user continues. Start with LLM preemption; enable preemptive TTS only after measuring wasted speech and false starts.

8.3 Filler / acknowledgement speech

LiveKit documents using `on_user_turn_completed()` to play a short filler phrase while the main answer is being generated. The filler should not be persisted as if it were the actual answer. Build a tiny separate "response bridge" vocabulary: "one sec", "let me check that", "yeah", "right", "okay, I see what you mean".

8.4 Speech style prompt

You are a conversational companion, not a narrator.

- Speak in short natural sentences.
- Prefer one idea per breath.
- Do not read headings, citations or markdown aloud.
- Do not repeat the user's entire question unless clarification is needed.
- Use brief acknowledgements naturally, but do not overuse them.
- Ask a follow-up only when it advances the conversation.
- When a tool is running, you may give one short status cue.
- If retrieved evidence is uncertain or conflicting, say so plainly.
- Never invent a memory, source or tool result.

9. Reference implementation skeleton

The following is a reference shape, not a blind copy-paste guarantee: plugin names and model-specific parameters should be checked against the exact package version pinned in your environment.

9.1 Python project shape

```
offline-buddy/
  pyproject.toml
  .env
  src/
    agent.py
    config.py
    retrieval/
      ingest.py
      search.py
      embeddings.py
      rerank.py
    memory/
      store.py
      recall.py
      write.py
    tools/
      permissions.py
      result_compactor.py
  audio/
```

```

    stt_adapter.py
    tts_adapter.py
data/
  raw/
  normalized/
  models/
infra/
  livekit/
  postgres/
  mcp/

```

9.2 Core AgentSession shape

```

from dotenv import load_dotenv
from livekit import agents
from livekit.agents import Agent, AgentServer, AgentSession
from livekit.agents import ChatContext, ChatMessage, TurnHandlingOptions, inference
from livekit.plugins import openai, silero

load_dotenv()
server = AgentServer()

class Buddy(Agent):
    def __init__(self):
        super().__init__(
            instructions=OPENING_PROMPT,
            tools=LOCAL_TOOLSETS,
        )

    async def on_user_turn_completed(
        self, turn_ctx: ChatContext, new_message: ChatMessage
    ) -> None:
        evidence = await retrieve_for_turn(new_message.text_content)
        if evidence:
            turn_ctx.add_message(
                role="assistant",
                content=render_private_evidence(evidence),
            )

@server.rtc_session(agent_name="offline-buddy")
async def offline_buddy(ctx: agents.JobContext):
    session = AgentSession(
        vad=silero.VAD.load(),
        turn_handling=TurnHandlingOptions(
            turn_detection=inference.TurnDetector(version="v1-mini"),
            preemptive_generation=True,
        ),
        stt=LOCAL_STT,
        llm=LOCAL_LLM,
        tts=LOCAL_TTS,
    )

    await session.start(room=ctx.room, agent=Buddy())

if __name__ == "__main__":
    agents.cli.run_app(server)

```

9.3 Local Ollama LLM

```

from livekit.plugins import openai

LOCAL_LLM = openai.LLM.with_ollama(
    model="qwen3.5:9b",
    base_url="http://127.0.0.1:11434/v1",
)

```

The LiveKit Ollama guide uses the OpenAI plugin with the Ollama Chat Completions endpoint. Swap the model tag for another local family without changing the rest of the LiveKit architecture.

9.4 Local vLLM LLM

```
from livekit.plugins import openai

LOCAL_LLM = openai.LLM(
    model="Qwen/Qwen3.8-27B",
    base_url="http://127.0.0.1:8000/v1",
    api_key="local-only",
)
```

vLLM currently exposes an OpenAI-compatible HTTP server and supports tool-calling configuration. Exact model IDs and parser flags must match the model being served.

9.5 Local Kokoro TTS

```
from livekit.plugins import openai

LOCAL_TTS = openai.TTS(
    model="kokoro",
    voice="af_alloy",
    api_key="not-needed",
    base_url="http://127.0.0.1:8880/v1",
    response_format="wav",
)
```

This is the current official LiveKit pattern for Kokoro-FastAPI.

9.6 Local Whisper-style STT

```
from livekit.agents import inference, stt
from livekit.plugins import openai

whisper = openai.STT(
    model="whisper-1",
    api_key="local-only",
    base_url="http://127.0.0.1:9000/v1",
    use_realtime=False,
)

# Because a Whisper-compatible endpoint may not stream natively:
# use VAD + StreamAdapter to turn buffered speech segments into STT requests.
LOCAL_STT = stt.StreamAdapter(
    whisper,
    inference.VAD(model="silero", min_silence_duration=0.5),
)
```

The exact endpoint implementation is yours. The important LiveKit rule is that non-streaming STT requires an explicit utterance-boundary mechanism; the official STT documentation demonstrates VAD + StreamAdapter for Whisper-style services.

10. Local deployment and startup

The supplied PDF proposes Docker Compose for the local stack. For current LiveKit development, the official self-hosting guide provides an even simpler path: install LiveKit Server and run `livekit-server --dev`. Use containers later when you want a reproducible production-like environment.

10.1 Start LiveKit locally

```
livekit-server --dev
```

The official local-development guide uses API key devkey and secret secret in dev mode and binds the signal server to 127.0.0.1:7880 by default.

10.2 Start Ollama

```
ollama serve
ollama pull qwen3.5:9b
ollama list
```

10.3 Start Kokoro-FastAPI

```
# Install/run the Kokoro-FastAPI project you have pinned locally.
# It should expose an OpenAI-compatible endpoint such as:
http://127.0.0.1:8880/v1
```

10.4 Start the buddy agent

```
uv sync
uv run python src/agent.py dev

# Console-only development test:
uv run python src/agent.py console
```

10.5 Local MCP

```
LOCAL_MCP_COMMAND="python infra/mcp/server.py"
```

Use MCPServerStdio for a local process, or MCPServerHTTP for a private local Streamable HTTP endpoint. Current LiveKit MCP docs describe Streamable HTTP as the modern transport and note SSE deprecation.

10.6 Suggested .env

```
LIVEKIT_URL=ws://127.0.0.1:7880
LIVEKIT_API_KEY=devkey
LIVEKIT_API_SECRET=secret
OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
KOKORO_BASE_URL=http://127.0.0.1:8880/v1
STT_BASE_URL=http://127.0.0.1:9000/v1
POSTGRES_DSN=postgresql://buddy:buddy@127.0.0.1:5432/buddy
MCP_BASE_URL=http://127.0.0.1:9100/mcp
```

11. Hardware planning and model-sizing

There is no single hardware requirement because the voice system runs several workloads concurrently. The supplied PDF suggests 16 GB RAM as a minimum and an NVIDIA GPU for sub-second behavior, but those are search-result estimates, not official LiveKit benchmarks. Current model catalogs give a better starting point for sizing by model artifact size.

Resource band	Practical configuration to test	What to watch
CPU-only / low-memory	Silero VAD + local turn detector v1-mini + small STT + Gemma4 E2B/E4B or Qwen3.5 4B + Piper/Chatterbox Nano.	Thermal throttling, STT segmentation, LLM time-to-first-token, TTS start delay.
12-24 GB RAM / one midrange GPU	Qwen3-ASR 0.6B/1.7B + Qwen3.5 9B or Gemma4 12B + Kokoro/Chatterbox Turbo + Qwen3-Embedding 0.6B.	GPU memory fragmentation and contention between LLM/STT/TTS.
24-48 GB RAM / stronger GPU	Qwen3.5 27B / Qwen3.8 27B / Gemma4 26B + larger STT/TTS + reranker.	Prompt cache/KV cache, concurrent model residency, background indexing.

Workstation / multi-GPU	35B-class or larger local LLM + larger ASR/TTS and multiple sessions.	Scheduling, model placement, batching and process isolation.
-------------------------	---	--

Model file size is not total memory usage: Ollama tags such as Qwen3.8 27B (~18 GB) or Gemma4 26B (~19 GB) describe the model artifact, not the full runtime footprint. Leave headroom for KV cache, Python, LiveKit, audio buffers, PostgreSQL and embedding/reranking processes.

If all models must be resident simultaneously

GPU memory / RAM budget $\approx$ LLM + STT + TTS + embedding/reranker + runtime + KV cache + safety headroom

A more practical deployment pattern is to keep the LLM resident, keep VAD resident, and run STT/TTS as separate long-lived services. Embedding/reranking can be loaded on demand or kept on CPU if throughput is modest.

12. Testing, evaluation and latency instrumentation

A near-human buddy needs a test suite that measures conversation quality, not just whether the model can answer a prompt.

Metric	How to measure	Why it matters
EOU latency	End-of-user-speech -> agent start.	Direct measure of turn-taking feel.
TTFT	First text token from LLM.	LLM responsiveness.
TTFA	First audio frame delivered.	What the user actually perceives.
Interruption reaction	User starts speaking while agent talks -> audio stop/recovery.	Natural conversational control.
Tool dwell	Tool invocation -> compact result available.	Detect slow MCP/RAG actions.
RAG grounding	Retrieved evidence -> answer trace.	Detect unsupported claims and retrieval misses.
Memory precision	Relevant memories recalled without unrelated memories surfacing.	Prevents "creepy" or distracting behavior.
False tool calls	Times tool used when plain response was enough.	Controls latency and reliability.
Tool-loop depth	Number of tool steps per user turn.	Controls runaway agent behavior.
TTS waste	Audio generated before an interruption that invalidates it.	Measures preemptive TTS cost/quality.

Conversation scenario suite

- Mid-sentence pause: "I think the real problem is..." then silence then continuation.
- Backchannel: "yeah", "uh-huh", "right" while the agent is talking.
- Self-correction: "Use SQLite - actually Postgres."
- Mixed RAG + action: "From my project notes, what did I decide? And add that to my task list."
- Ambiguous reference: "What did I say about that yesterday?"
- Large source: ask about a specific page/section in a long PDF.
- Conflicting sources: two attached documents disagree.
- No evidence: ask a question not supported by attachments.
- MCP failure: server unavailable or tool returns an error.
- Slow tool: simulate a 2-second API and test filler speech.
- Language switching: English/Hindi mixed utterance if your STT and TTS stack supports it.

- Long conversation: 50+ turns and verify latency does not climb with history size.

Trace every turn

```
session_id
turn_id
user_audio_start / end
VAD state
turn_detector decision
STT first partial / final
RAG query + selected sources
LLM first token / tool call
MCP duration + result size
LLM final token
TTS first audio
interruption events
final transcript
```

13. Migration from the supplied PDF

Supplied PDF approach	Current update	Action
livekit-agents + silero + OpenAI plugin	Still valid as the integration strategy, but use current 1.8.x APIs and pin versions.	Start from current official Python starter structure, then replace provider objects with local endpoints.
Silero VAD	Still directly relevant; current LiveKit docs recommend it with the turn detector.	Keep it.
Whisper local STT	Still valid conceptually.	Wrap a local Whisper-compatible service or use a custom STT. Add StreamAdapter when endpoint is non-streaming.
Ollama + llama3	Still valid conceptually.	Keep Ollama but test current local models such as Qwen3.5/Gemma4/Qwen3.6/Qwen3.8 as candidates.
Kokoro via OpenAI-compatible API	Still current and officially documented.	Use the current Kokoro-FastAPI + OpenAI adapter pattern.
Docker Compose for everything	Useful but not mandatory for dev.	Use livekit-server --dev for simplest local LiveKit; use Compose for reproducible deployment.
Basic function tools	Current tool system is richer.	Use toolsets, filtered MCPToolset, bounded tool steps and result compaction.
Simple VAD endpointing	Not enough for the desired buddy.	Add the current audio turn detector v1-mini plus interruption tuning.

What should NOT be copied blindly from an older tutorial

- Old package versions pinned around ~1.5 if the goal is a current implementation.
- Deprecated MCP server parameters or SSE-only assumptions.
- The idea that increasing max iterations fixes tool loops.
- A giant prompt containing every document instead of retrieval.
- Treating all TTS engines as interchangeable without matching their streaming/protocol semantics.
- Assuming a realtime speech-to-speech model is automatically better for RAG + MCP. The current LiveKit docs still position the STT-LLM-TTS pipeline as the mature general-purpose route.

14. Build roadmap

Stage	Build	Acceptance test
0. Infrastructure	Local LiveKit Server + browser client + AgentServer.	Two-way audio works on LAN.
1. Voice baseline	Silero VAD + local STT + local LLM + Kokoro/Piper.	10-turn conversation without cloud dependency.
2. Human turn-taking	v1-mini audio turn detector + interruption tuning + preemptive generation.	Mid-sentence pauses and user interruptions behave naturally.
3. Data attachment	PDF/text ingestion + OCR + Qwen3 embeddings + pgvector + RAG hook.	Questions about attached files are grounded in the correct source.
4. Memory	Episodic/profile memory + user-controlled memory writes.	Buddy can recall selected prior context without context-window growth.
5. MCP	Local tools with filtering, permissions and tool-loop bounds.	Read tools work; write tools require appropriate confirmation.
6. Rich multimodal data	Audio/video/code/spreadsheets.	Each source type preserves provenance and retrieval quality.
7. Evaluation	Automated transcript/latency tests + session traces.	Regression suite catches turn-taking, grounding and tool regressions.
8. Optimization	Model residency, quantization, process placement, streaming and caching.	Measured improvement in TTFA/EOU without losing grounding or reliability.

Recommended first end-to-end stack

```

LiveKit Server (self-hosted)
+ AgentServer / AgentSession (Python, 1.8.x line)
+ Silero VAD
+ LiveKit audio TurnDetector v1-mini
+ Qwen3-ASR 0.6B or local Whisper wrapper
+ Ollama (Qwen3.5 9B or Gemma4 12B)
+ Kokoro-FastAPI
+ PostgreSQL + pgvector
+ Qwen3-Embedding-0.6B + optional Qwen3-Reranker-0.6B
+ local MCP servers (stdio or Streamable HTTP)

```

This is a baseline to implement and measure, not a claim that one exact combination is universally optimal.

15. Troubleshooting matrix

Symptom	Likely cause	Fix
Agent answers too early	VAD endpointing too aggressive or turn detector unavailable.	Verify v1-mini is active; increase minimum silence/endpointing delay within reason.
Agent cuts user off	Interruption threshold too sensitive.	Tune interruption detection and false-interruption recovery.
Long silence before response	STT, RAG, LLM or TTS serial latency.	Measure EOU/TTFT/TTFA; use preemptive generation and short filler for long operations.
Whisper only responds after pauses	Endpoint is non-streaming.	Use VAD + StreamAdapter or choose a streaming ASR server.
Kokoro outputs fail	OpenAI-compatibility mismatch.	Pin Kokoro-FastAPI + LiveKit versions; verify /v1/audio/speech response and audio format.
MCP loops	Tool set too broad or tool descriptions ambiguous.	Filter tools; use stronger schemas; cap tool steps; compact results.
RAG answer ignores source	Retrieval misses or evidence is buried.	Inspect retrieved chunks; rerank; reduce injected context; add source labels.
Memory feels wrong	Too many automatic memories.	Use conservative memory writes; require

		explicit or repeated facts for durable profile memory.
GPU runs out of memory	Several models loaded simultaneously + KV cache.	Use smaller models, separate services, quantization and/or staged loading.
Offline system secretly calls cloud	Inference defaults or third-party plugins remain enabled.	Audit every endpoint and environment variable; disable LiveKit Inference and cloud-only noise cancellation/plugins.

Appendix A. Example schemas

Source and chunk schema

```
source(id, persona_id, uri, source_type, title, language, checksum,
      created_at, updated_at, permissions, ingestion_status)

chunk(id, source_id, parent_chunk_id, ordinal, text,
      page, section, start_ms, end_ms, code_symbol, metadata_json,
      embedding_model, embedding)

memory(id, persona_id, kind, text, source_turn_id,
      created_at, last_validated_at, confidence, status, embedding)
```

Retrieval result object

```
{
  "source_id": "src_8f3a",
  "title": "Cloud architecture notes",
  "location": "page 14 / section 3.2",
  "score": 0.91,
  "text": "...",
  "freshness": "document",
  "permissions": "persona:buddy_01"
}
```

MCP permission sketch

```
tool_policy:
  read_tools:
    - search_project
    - read_document
    - list_tasks
  write_tools:
    - add_task
    - create_event
  confirmation_required:
    - delete_*
    - send_*
    - publish_*
```

Appendix B. Official source index

LiveKit Agents release page - Current repository release list; 1.8.2 is shown as latest.

<https://github.com/livekit/agents/releases>

LiveKit Voice AI quickstart - Current AgentServer/AgentSession starter architecture and current Python requirements.

<https://docs.livekit.io/agents/start/voice-ai/>

LiveKit models overview - Current model families and OpenAI-compatible integration patterns.

<https://docs.livekit.io/agents/models/>

LiveKit turn-taking overview - Current turn detector and turn handling architecture.

<https://docs.livekit.io/agents/logic/turns/>

LiveKit turn detector - v1/v1-mini behavior, local CPU v1-mini, supported languages.

<https://docs.livekit.io/agents/logic/turns/turn-detector/>

LiveKit turn tuning - Endpointing, interruptions and preemptive generation.

<https://docs.livekit.io/agents/logic/turns/tuning/>

LiveKit RAG / external data - Current RAG insertion patterns and external data guidance.

<https://docs.livekit.io/agents/logic/external-data/>

LiveKit pipeline nodes - Current on_user_turn_completed() hook and pipeline extension points.

<https://docs.livekit.io/agents/logic/nodes/>

LiveKit MCP - MCPToolset, filtering, HTTP/stdio and modern transport guidance.

<https://docs.livekit.io/agents/logic/tools/mcp/>

LiveKit toolsets - Toolset grouping and dynamic discovery.

<https://docs.livekit.io/agents/logic/tools/toolsets/>

LiveKit self-hosting - Self-hosting overview and private infrastructure model.

<https://docs.livekit.io/transport/self-hosting/>

LiveKit local server - Current livekit-server --dev local setup.

<https://docs.livekit.io/transport/self-hosting/local/>

LiveKit Ollama - Local Ollama integration.

<https://docs.livekit.io/agents/models/llm/ollama/>

LiveKit Kokoro - Official local Kokoro-FastAPI integration.

<https://docs.livekit.io/agents/models/tts/kokoro/>

LiveKit STT overview - Current STT architecture and Whisper/StreamAdapter note.

<https://docs.livekit.io/agents/models/stt/>

Qwen3.5 Ollama catalog - Current local Qwen3.5 sizes and context catalog.

<https://ollama.com/library/qwen3.5>

Qwen3.6 Ollama catalog - Current local Qwen3.6 sizes and context catalog.

<https://ollama.com/library/qwen3.6>

Qwen3.8 Ollama catalog - Current local Qwen3.8 27B catalog.

<https://ollama.com/library/qwen3.8>

Gemma 4 Ollama catalog - Current local Gemma 4 sizes and context catalog.

<https://ollama.com/library/gemma4>

Mistral Small 3.2 Ollama catalog - Current local Mistral Small 3.2 capabilities.

<https://ollama.com/library/mistral-small3.2>

Qwen3 function calling docs - Current official Qwen tool-calling guidance.

https://github.com/QwenLM/Qwen3/blob/main/docs/source/framework/function_call.md

Qwen3-ASR repository - Current local ASR models, languages and streaming/vLLM paths.

<https://github.com/QwenLM/Qwen3-ASR>

NVIDIA NeMo Speech - Current local ASR/TTS framework updates, including Parakeet and Nemotron Speech.

<https://github.com/NVIDIA-NeMo/Speech>

Qwen3 Embedding model card - Current Qwen3 embedding/reranker sizes and dimensions.

<https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>

Piper repository - Current local TTS engine and ongoing 2026 updates.

<https://github.com/OHF-Voice/piper1-gpl>

Chatterbox repository - Current Chatterbox Turbo/Nano/V3 model family and multilingual coverage.

<https://github.com/resemble-ai/chatterbox>

F5-TTS repository - Current local TTS implementation and 2026 release history.

<https://github.com/SWivid/F5-TTS>

Source note for the supplied PDF

The attached six-page PDF is a saved Google Search/AI Mode result titled "can we use livekit with all offline model to create a voice agent". Its useful baseline is the four-pillar offline stack: Silero VAD, Whisper-family STT, Ollama/vLLM LLMs, and Kokoro/local TTS, plus the proposed multi-container setup and Python agent. Its hardware figures (for example, 16 GB RAM minimum and 2-4 second CPU latency) are presented in the search result as estimates, so this guide does not treat them as authoritative benchmarks.

End state

A practical mental model: The buddy is a realtime shell around a local intelligence stack. Keep the realtime loop small and predictable; let retrieval, memory and MCP services do specialized work outside the audio path; return compact evidence/results to the LLM; then speak naturally.